

Matrix completion and application to movies recommendation system

Cathérine MONOUE KONGA, Sepideh MALEKI ROUDPOSHTI

Master in Financial mathematics

University of Luxembourg

Group 2

May 9, 2026

Introduction

Matrix completion is a fundamental problem in modern applied mathematics and data science. It consists in recovering the missing entries of a partially observed matrix by exploiting an underlying low-rank structure. This problem appears naturally in many real-world applications such as image reconstruction, recommendation systems and signal processing [1]. In particular, recommendation systems often rely on incomplete user–item preference matrices, where only a small subset of ratings is known. Recovering the missing entries then becomes essential in order to provide accurate predictions and personalized recommendations.

The objective of this project, carried out in the context of the Linear Algebra course, is to explore the mathematical foundations of the matrix completion problem and to study different approaches used to solve it. We first introduce the theoretical framework of matrix completion and discuss why low-rank structures play a central role in the recovery process. We then present several optimization approaches, including rank minimization, nuclear norm minimization, and matrix factorization methods.

To illustrate these concepts on a practical problem, we apply matrix completion techniques to movie rating data from the MovieLens dataset. In this setting, the ratings given by users to movies form a partially observed matrix with many missing entries. Our goal is therefore to predict these missing ratings by reconstructing the complete matrix from the available observations. We compare different approaches for this task and discuss their performance and practical limitations.

Contents

1	Mathematical framework for matrix completion	1
1.1	Matrix completion problem	1
1.2	Optimization Approaches for Matrix Completion	1
1.2.1	Rank minimization	1
1.2.2	Nuclear norm minimization	2
1.3	Matrix factorization	5
2	Applications to movie ratings	6
2.1	Dataset	6
2.2	Prediction of missing ratings	7
2.2.1	Training and test procedure	7
2.2.2	Nuclear norm minimization	7
2.2.3	Matrix factorisation	8
2.2.4	Discussion	9

1 Mathematical framework for matrix completion

1.1 Matrix completion problem

Let $M \in \mathbb{R}^{n \times m}$ be a matrix such that only a subset of its entries is observed. We want to find the missing entries based on what has been observed. To achieve this, we seek values that minimize the discrepancy between the observed and the predicted entries. Also, we will assume that, the original matrix is of low rank. This additional condition on the structure of M means that the matrix is not completely arbitrary [1]. In other words, its rows and columns are related through a small number of hidden factors. As a result, even though the observed matrix contains much less information than the full matrix, the recovery from partial observations is still possible.

1.2 Optimization Approaches for Matrix Completion

In this section, we present three methodologies used for matrix completion: rank minimization, nuclear norm minimization, and matrix factorization. To this end, we introduce the sampling operator and dual norm, defined as follows:

Definition: Sampling operator

Let Ω be the set of observed indices. We define the sampling operator P_Ω as:

$$(P_\Omega(X))_{i,j} = \begin{cases} X_{i,j} & \text{if } (i,j) \in \Omega \\ 0 & \text{otherwise} \end{cases}$$

Definition: Dual norm

For a given norm $\|\cdot\|$, we call dual norm the quantity:

$$\|\mathbf{X}\|_D := \max\{\langle \mathbf{X}, \mathbf{Y} \rangle : \|\mathbf{Y}\| \leq 1\}$$

Remark 1. *The dual norm of the nuclear norm is the spectral norm.*

1.2.1 Rank minimization

Problem Statement

Let $M \in \mathbb{R}^{n \times m}$ be the original matrix and let Ω denote the set of observed indices. The rank minimization problem for matrix completion is defined as follows:

$$\min_X \text{rank}(X), \quad \text{s.t.} \quad P_\Omega(X) = P_\Omega(M).$$

NP-hardness of rank minimization The rank minimization problem is known to be NP-hard. This means that there is no known efficient algorithm that can solve it in polynomial time for large-scale problems.

In practice, this implies that directly minimizing the rank of a matrix is computationally intractable when the matrix size is large.

Therefore, instead of solving the rank minimization problem directly, a common approach is to replace it with nuclear norm minimization, which is a convex and tractable relaxation of the original problem. Because rank minimization is too difficult to solve directly, we replace it with nuclear norm minimization. This new problem is easier to solve, but it still keeps the same main idea: we look for a matrix with low rank that matches the observed entries.

1.2.2 Nuclear norm minimization

Definition: nuclear norm

The nuclear norm of $\mathbf{X}$ is :

$$\|\mathbf{X}\|_{\star} : \sum_{i=1}^n \sigma_i(\mathbf{X})$$

where σ_i is the i^{th} largest singular value

1.2.2.1 Problem Statement

Let $M \in \mathbb{R}^{n \times m}$ be the original matrix and let Ω be the set of observed indices. The matrix completion problem via nuclear norm minimization aims to recover X by solving

$$\min \|X\|_{\star}, \quad \text{s.t.} \quad P_{\Omega}(X) = P_{\Omega}(M). \quad (1)$$

Unlike the rank function, the nuclear norm is convex, which makes the optimization problem tractable.

By definition of the nuclear norm, a natural approach to solving problem (1) is to search for matrix whose singular values are as small as possible. This idea was developed by Beck and Teboulle [2], who proposed the Fast Iterative Shrinkage-Thresholding Algorithm (FISTA).

To apply FISTA, the constrained problem is reformulated as an unconstrained optimization problem using a least-squares error term and sampling operator:

$$\min \left(\frac{1}{2} \|P_{\Omega}(X) - P_{\Omega}(M)\|_F^2 + \tau \|X\|_{\star} \right). \quad (2)$$

We evaluate the quality of the reconstruction with the relative recovery error:

$$\text{Recovery error} = \frac{\|\hat{X} - M\|_F}{\|M\|_F},$$

where M is the original matrix and $\hat{X}$ is the reconstructed matrix. This metric tells us how far the recovered matrix is from the true matrix overall. A smaller value indicates better recovery.

1.2.2.2 Fast Iterative Shrinkage-Thresholding Algorithm

We consider optimization problems of the form

$$\min_{x \in \mathbb{R}^n} F(x) = f(x) + g(x),$$

where f is a smooth convex function with Lipschitz continuous gradient and g is a convex (possibly non-smooth) function. The main idea of iterative shrinkage methods is to avoid solving this problem directly by approximating it at each iteration with a simpler subproblem.

Starting from a point y_k , which combines the current estimate and information from previous iterations, the smooth function f is locally approximated by a quadratic function. This leads to the subproblem:

$$\min_x Q_L(x, y_k) = \min_x \left(f(y_k) + \langle x - y_k, \nabla f(y_k) \rangle + \frac{L}{2} \|x - y_k\|^2 + g(x) \right),$$

where L is the lipschitz constant of ∇f

The solution of this problem

$$D_{g/L}(y_k) = \arg \min_x Q_L(x, y_k)$$

A. Beck and M. Teboulle [2] show that this solution is equivalent to:

$$D_{g/L}(v) = \arg \min_x \left(g(x) + \frac{L}{2} \|x - v\|^2 \right), \text{ with } v = y_k - \frac{1}{L} \nabla f(y_k).$$

Here, $D_{g/L}(v)$ denotes the proximal operator of g . This operator balances two goals: staying close to v while enforcing the structure induced by g .

The overall update can thus be interpreted as a gradient-descent step on f followed by a proximal step that enforces a low-rank structure. The next iterate is defined as :

$$x_{k+1} = D_{g/L} \left(y_k - \frac{1}{L} \nabla f(y_k) \right).$$

To accelerate convergence, a momentum term is introduced, which controls how much information from previous iterations is reused:

$$y_{k+1} = x_{k+1} + \frac{t_k - 1}{t_{k+1}} (x_{k+1} - x_k).$$

At the first iteration, the momentum is equal to 1 since no previous information is available.

1.2.2.3 Application of FISTA on nuclear norm minimization problem

In the context of problem (2), we identify

$$f(X) = \frac{1}{2} \|P_\Omega(X) - P_\Omega(M)\|_F^2, \quad g(X) = \tau \|X\|_\star.$$

The gradient of f is given by: $\nabla f(X) = P_\Omega(X - M)$.

In fact, since the projection operator P_Ω is linear, its derivative at any point X in the direction H satisfies $D(P_\Omega)(X)[H] = P_\Omega(H)$. In other words, the derivative of the projection operator is the operator itself. Therefore, the gradient simplifies to

$$\nabla f(X) = P_\Omega(X - M).$$

For the proximal operator, we use the Singular Value Thresholding (SVT) operator below, which provides a closed-form solution of the nuclear norm minimization problem (2):

$$D_\tau(X) = U\Sigma'V^\top,$$

where $\Sigma' = \text{diag}(\sigma'_1, \sigma'_2, \dots)$ and

$$\sigma'_k = \begin{cases} \sigma_k - \tau, & \text{if } \sigma_k \geq \tau, \\ 0, & \text{if } \sigma_k < \tau. \end{cases}$$

where $X = U\Sigma V^\top$ is the SVD of X , with $\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r)$.

The full procedure of FISTA applied to nuclear norm minimization is summarized in the algorithm below.

Algorithm 1 FISTA for Low-Rank Matrix Estimation

- 1: **Input:** T, L, τ
 - 2: **Initialization:** $R_1 = 0$ (initial estimate), $t_1 = 1$ (initial momentum parameter)
 - 3: **for** $k = 1$ to T **do**
 - 4: $X_k \leftarrow D_{\tau/L}(R_k - \frac{1}{L}P_\Omega^*(P_\Omega(R_k) - y))$
 - 5: $t_{k+1} \leftarrow \frac{1 + \sqrt{1 + 4t_k^2}}{2}$
 - 6: $R_{k+1} \leftarrow X_k + \frac{t_k - 1}{t_{k+1}}(X_k - X_{k-1})$
 - 7: **end for**
 - 8: **Output:** X_T
-

Remark: In the context of the movie rating problem, the SVD decomposition makes sense with the following framework:

- Both users and movies can be described using a small number of hidden features (for example: action, romance, comedy, famous actors).
- Each row of U describes what a user likes (e.g., likes action a lot, dislikes romance).
- Each row of V describes what a movie is like (e.g., a lot of action, slightly funny, horror).
- The matrix Σ tells us which features are more important overall.
- The rating of a user for a movie is interpreted as the linear combination between the user's preference vector and the movie's feature vector in the hidden features space.

1.2.2.4 Existence of Solutions for Nuclear Norm Minimization

In this part, we present a result that address the existence of solutions to the nuclear norm minimization problem.

Definition: Matrix Coherence

Let U be a subspace of $\mathbb{R}^n$ of dimension r , and let P_U be the orthogonal projection onto U . Then

the coherence of U (with respect to the standard basis (e_i)) is defined as

$$\mu(U) \equiv \frac{n}{r} \max_{1 \leq i \leq n} \|P_U e_i\|_2^2.$$

This quantity gives an idea about the distribution of the data and is used to determine if we can recover a matrix or not. Low coherence means that the information is spread across the matrix, which makes recovery possible, as shown by Candès-Recht [3].

Theorem: Candès-Recht Recovery Conditions

Let M be an $n_1 \times n_2$ matrix of rank r satisfying assumptions states below and let $n = \max(n_1, n_2)$.

Assumptions:

1. The coherences satisfy $\max(\mu(U), \mu(V)) \leq \mu_0$ for some positive constant μ_0 .
2. The $n_1 \times n_2$ matrix $\sum_{k=1}^r u_k v_k^*$ has a maximum entry bounded in absolute value by $\mu_1 \sqrt{\frac{r}{n_1 n_2}}$, for some positive constant μ_1

Suppose we observe m entries of M whose locations are sampled uniformly at random. Then there exist constants $C, c > 0$ such that if

$$m \geq C \max(\mu_1^2, \mu_0 \mu_1, \mu_0 n^{1/4}) nr(\beta \log n) \quad (1.9)$$

for some $\beta > 2$, then the minimizer of problem (1.5) is unique and equal to M with probability at least

$$1 - cn^{-\beta}.$$

Moreover, if $r \leq \mu_0^{-1} n^{1/5}$, this estimate can be improved to

$$m \geq C \mu_0 n^{6/5} r(\beta \log n), \quad (1.10)$$

with the same probability of success.

Meaning: This theorem says that if the coherence is low, we can still recover the full matrix with few samples.

1.3 Matrix factorization

problem statement Matrix factorization is a practical approach to matrix completion. Instead of recovering the full matrix directly, we approximate the rating matrix by the product of two low-dimensional matrices:

$$M \approx UV^T,$$

where $U \in \mathbb{R}^{n \times f}$ contains the latent factors of users and $V \in \mathbb{R}^{m \times f}$ contains the latent factors of movies. Here, n is the number of users, m is the number of movies, and f is the number of latent factors, with $f \ll n, m$.

In this implementation, we use a basic matrix factorization model without bias terms. The predicted rating of user i for movie j is given by

$$\hat{r}_{ij} = U_i V_j^T.$$

The model parameters are learned from the observed ratings by minimizing a regularized squared error on the training set:

$$\min \sum_{(i,j) \in \kappa} (r_{ij} - \hat{r}_{ij})^2 + \lambda (\|U_i\|^2 + \|V_j\|^2),$$

where κ is the set of observed user–movie pairs and $\lambda > 0$ is a regularization parameter.

To learn the model parameters, we use a stochastic gradient descent type procedure. At each step, for one observed pair (i,j) , we compute the prediction error

$$e_{ij} = r_{ij} - \hat{r}_{ij},$$

and then update the corresponding user and movie latent vectors as follows:

$$U_i \leftarrow U_i + \gamma (e_{ij} V_j - \lambda U_i),$$

$$V_j \leftarrow V_j + \gamma (e_{ij} U_i - \lambda V_j),$$

where $\gamma > 0$ is the learning rate.

In practice, the observed ratings are first divided into a training set and a test set. In our implementation, one rating for each movie is first forced into the training set, so that every movie appears at least once in training. Then, the remaining ratings are split user-wise into training and test sets. After that, the user IDs and movie IDs are converted into numerical indices, and the latent factor matrices are initialized with small random values.

After training, the learned matrices U and V are used to estimate missing entries of the user–movie matrix. The quality of the predictions can then be evaluated on the test set, for example by using the recovery error defined above.

2 Applications to movie ratings

2.1 Dataset

The data used in this project are taken from the MovieLens ¹ dataset. We chose the dataset recommended for education and development. The dataset contains 100 836 ratings given by 610 users on 9 724 movies. The MovieLens dataset provides four main files, which are described as follows:

- **Movies.csv**: contains information about the movies, such as titles and genres.
- **Ratings.csv**: contains 100836 ratings. Each entry includes a user ID, a movie ID, a rating value, and a timestamp.
- **Tags.csv**: contains user-generated comments associated with movies.
- **Links.csv**: provides identifiers that can be used to link this dataset to other movie data sources.

¹<https://grouplens.org/datasets/movielens/>

Among these files, the main file used in our implementations is `Ratings.csv`, since it provides the observed user–movie ratings needed to build the prediction models. In particular, the same rating data are used in both matrix factorization and nuclear norm minimization, although the representation of the data differs in the two approaches. For matrix factorization, the data are handled as observed user–movie–rating triples, while for nuclear norm minimization they are rearranged into a user–movie matrix. The file `Movies.csv` provides additional descriptive information, but it is not directly used in the core numerical implementation.

2.2 Prediction of missing ratings

2.2.1 Training and test procedure

Definition

We denote by *test set* ratings that are treated as missing during the training phase and are used only to evaluate the model.

The training set consists of the ratings considered as observed data, which are used to learn and reconstruct the original matrix.

The test error is the relative error computed only on the ratings belonging to the test set. This measure helps us evaluate how well the model generalizes to unseen data.

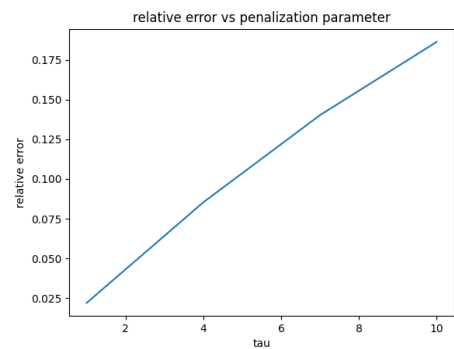
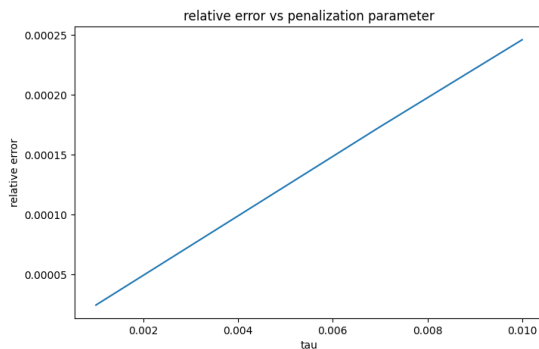
The training error is the error computed on the ratings belonging to the training set.

Procedure

We split our dataset into two parts: a training set which has 60% of the overall ratings and a test set. For each algorithm, we learn the matrix using only the ratings in the training set, and then we evaluate the predictions on the test set in order to assess the quality of the reconstruction.

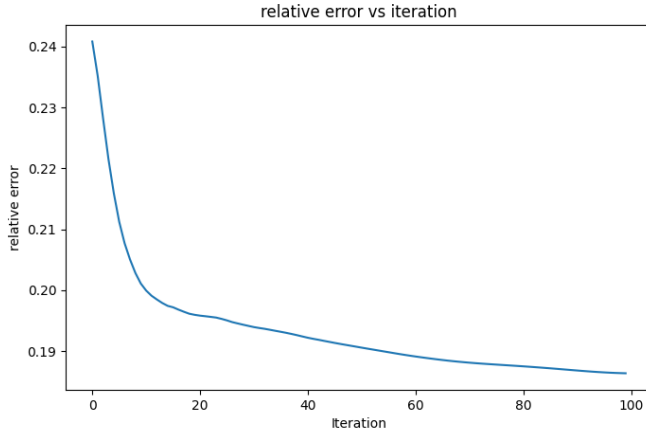
For nuclear norm minimization, we test different values of the penalization parameter in order to choose the best one. Whereas for matrix factorization, we test different values of the rank and observe how the training error evolves in order to select an appropriate rank for the model.

2.2.2 Nuclear norm minimization



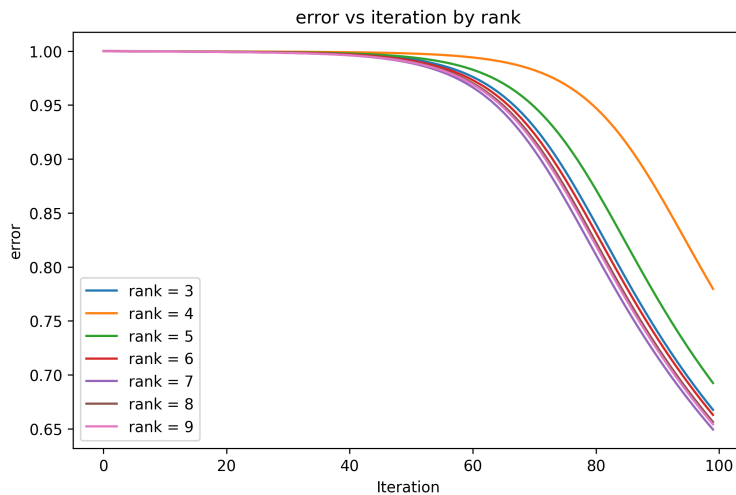
This graph shows that smaller values of the penalization parameter generally give better performance on the training set. However, when we choose a very small value such as $\tau = 0.001$, the training error becomes extremely small $2 \cdot 10^{-5}$, while the test error is very high 0.99. This indicates that the model is overfitting: it fits the training data too well but does not generalize to unseen data.

To address this issue, we increase the penalization parameter and choose $\tau = 10$. In this case, the training error is higher than for small values of τ , but the test error decreases significantly to 0.29. The figure below shows the evolution of the error for this model over iterations



We observe that the error decreases as the number of iterations increases. This indicates that the model improves during optimization, unlike the case of a very small τ , where additional iterations led to worse performance. The resulting predicted matrix has rank 64, suggesting that a more regularized and less complex model leads to better generalization.

2.2.3 Matrix factorisation



The figure above presents the evolution of the relative training error for the matrix factorization model. We observe that the model improves as the rank increases up to $r=7$, indicating effective learning in this range. However, for ranks larger than 7, the performance deteriorates, and the relative error increases, suggesting overfitting or loss of generalization. This behavior suggests that the underlying matrix likely has an intrinsic rank close to 7, as this value consistently yields the lowest error across iterations.

For $r=7$, the matrix factorization algorithm achieves a final training relative error of 0.64. When evaluated on the test set, the recovery error is 0.63, which is very close to the training error. This similarity indicates that the model generalizes reasonably well and captures meaningful structure in the data.

2.2.4 Discussion

comparison between Matrix factorization and nuclear norm minimization

As suggested in the literature [1], nuclear norm minimization is a powerful approach for matrix completion and produces good results. However, it is also known to be computationally expensive, as it requires repeated Singular Value Decomposition (SVD) [4], especially for large matrices, something we also observed in our experiments. By contrast, matrix factorization is easier to implement and computationally lighter in practice. In our experiments, it produced stable results and achieved similar training and test errors, suggesting that the model generalized reasonably well to the movie rating data.

Limitations

As in any project, we encountered several difficulties, particularly with nuclear norm minimization. In comparison, matrix factorization was more straightforward to implement and did not raise the same memory issues. This made it easier to experiment with different ranks and observe how the model behaved during learning. When trying to implement our own SVT (Singular Value Thresholding) algorithm without relying on Python’s built-in functions, we faced memory issues. These problems are likely due to numerical approximations and the cost of repeated matrix decompositions.

For this reason, we decided to replace our custom SVT implementation with a built-in Python function to improve stability and efficiency. Additionally, we neither verified that the matrix coherence was small nor carefully tuned the parameters to enable comparison of the time convergence of the two models.

Conclusion

In conclusion, this project allowed us to explore the matrix completion problem from both a theoretical and practical perspective. We investigated two main approaches for solving the problem: nuclear norm minimization and matrix factorization. These methods were then applied to a real-world movie rating dataset in order to predict missing user ratings.

Our experiments showed that both approaches are capable of recovering missing information and producing reasonable predictions. Nuclear norm minimization gives good reconstruction performance, in agreement with the literature, but at the cost of higher computational complexity due to repeated singular value decompositions. On the other hand, matrix factorization proved to be easier to implement and more computationally efficient.

This project also highlighted several practical challenges related to numerical implementation, memory consumption, and parameter selection. In particular, implementing the SVT algorithm

from scratch emphasized the importance of stable numerical routines and efficient linear algebra tools in large-scale optimization problems.

References

- [1] Y. Chen and Y. Chi. “Harnessing Structures in Big Data via Guaranteed Low-Rank Matrix Estimation”. In: *IEEE Signal Processing Magazine* 35.4 (2018), pp. 14–31.
- [2] A. Beck and M. Teboulle. “A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems”. In: *SIAM Journal on Imaging Sciences* 2.1 (2009), pp. 183–202.
- [3] E. J. Candès and B. Recht. “Exact Matrix Completion via Convex Optimization”. In: *Foundations of Computational Mathematics* 9.6 (Dec. 2009), pp. 717–772.
- [4] X. P. Li et al. “A Survey on Matrix Completion: Perspective of Signal Processing”. In: *arXiv preprint arXiv:1901.10885* (2019).
- [5] Y. Koren, R. Bell, and C. Volinsky. “Matrix factorization techniques for recommender systems”. In: *Computer* 42.8 (2009), pp. 30–37.
- [6] Y. Chen. “Low-Rank Matrix Recovery”. In: *ELE 520: Mathematics of Data Science, Princeton University, Fall 2020* Lecture slides (2020).
- [7] V. Morgenshtern. “Lecture 19–20: Matrix Completion”. In: *Mathematical Methods for Machine Learning and Signal Processing, SS 2019* Lecture notes (2019).